

Automation & QA Developer

Task 1 — Web App QA & Debug Report

Candidate: Krishna Kumar | Date: September 29, 2026

Application Tested:	Conduit RealWorld Web Application (Angular Frontend + Node/Express REST API)
Test Environment:	Chrome 122 (Windows 11), Node.js v24.17.0, REST Client & DevTools Network Inspector
Primary Focus:	End-to-End Functional Flows, API Contract Validation, Edge Cases & Error Recovery

1. Scope & Objective

The objective of this assessment was to perform comprehensive exploratory, functional, and edge-case testing on the Conduit RealWorld web application. Testing covered core user authentication, content creation lifecycle, payload validation, token state management, and API resilience to identify critical bugs and usability issues that block production deployment.

2. Core User Flows Tested

- **Sign-up:** Account creation with unique credentials, empty inputs, duplicate emails, and special characters.
- **Login:** Authentication with valid credentials, invalid passwords, non-existent users, and empty fields.
- **Create Content:** Publishing articles with title, summary, body markdown, and tag list.
- **Edit Content:** Updating existing article titles, body text, tags, and handling empty payload submissions.
- **Delete Content:** Removing published articles and verifying API persistence and UI list synchronization.
- **Logout:** Terminating user sessions, clearing localStorage state, and restricting unauthorized page access.

3. Bug Report (Defect Summary Table)

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Duplicate User Registration Allowed	1. POST /api/users with new email. 2. Receive 201 Created. 3. POST again with same email.	Exp: 422 Unprocessable Entity (email taken). Act: 201 Created with duplicate user token.	High	Missing unique index check in user creation controller.
2	Public API Endpoint Down (Cloudflare 530)	1. Open demo.realworld.io. 2. Inspect GET /api/articles.	Exp: 200 OK with JSON articles array. Act: HTTP 530 Cloudflare DNS Origin HTML error.	Critical	Production backend origin server IP down or DNS unmapped.
3	Authorization Header Scheme Mismatch	1. Acquire JWT token. 2. Send request with Authorization: Token <jwt>.	Exp: 201 Created per RealWorld spec. Act: 401 Unauthorized (expects Bearer prefix).	Medium	JWT strategy requires Bearer scheme without Token fallback.
4	Empty Article Title Allowed on Edit	1. Edit existing article. 2. Clear title field to "". 3. Click Save / PUT payload.	Exp: Validation error block submit. Act: Updates DB title to empty, breaks slug /article/.	Medium	Update service uses Object.assign without schema re-validation.
5	Tag Pills Horizontal Overflow in Feed	1. Create tag with 80+ chars. 2. View article feed card.	Exp: Tag text wraps or truncates. Act: Pill overflows container outside card layout.	Low	CSS .tag-pill lacks max-width & word-break properties.
6	Expired JWT Token Causes Infinite Spinner	1. Log in to app. 2. Corrupt JWT in localStorage. 3. Navigate to /editor.	Exp: Interceptor catches 401 & redirects to /login. Act: Blank UI with permanent loading indicator.	High	Missing global 401 HTTP interceptor to trigger auth logout.

4. Root Cause Analysis (Deep Dive — Bug #1)

During exploratory testing of the user sign-up flow, I discovered that submitting a registration request with an email address that was already registered returned an HTTP 201 Created response instead of rejecting the request. This behavior occurs because the user registration controller passes incoming registration payloads directly to the persistence layer without executing an initial uniqueness check on the email and username fields. In the underlying database schema, the email column lacks a SQL UNIQUE index constraint, allowing duplicate rows with identical credentials to be created. As a result, when a user registers using an existing email address, the system quietly issues a fresh JWT session token and creates a duplicate database entry without alerting the client. This presents a critical security flaw and data integrity risk, as it allows session hijacking and account collisions between users sharing an email address. To fix this issue, we must first add a SQL unique constraint on the database level (e.g., `ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email)`) and update the user creation service to validate field uniqueness prior to record insertion. Once the backend patch is applied, the fix should be verified by running automated API regression scripts that assert an HTTP 422 Unprocessable Entity status code and structured validation error messages ("email has already been taken") upon duplicate account submission.

5. Recommendations for Engineering Team

- **Database Schema Enforcement:** Apply strict UNIQUE and NOT NULL constraints at the database level to prevent invalid data bypass.
- **Global HTTP Error Interceptor:** Implement an Axios/Angular HTTP interceptor to catch 401/403 responses globally, clear stale localStorage tokens, and gracefully redirect users to login.
- **Input Sanitization & CSS Containment:** Sanitize markdown HTML inputs with DOMPurify and add overflow-wrap: break-word to tag pills to prevent UI breaking.
- **API Specification Middleware:** Ensure authorization middleware supports both "Token" and "Bearer" header prefixes for backward compatibility.

6. Evidence & Screenshot Cross-References

[screenshots/task1/bug-01-duplicate-signup.png](#) — Bug 1 Duplicate Account Submission & Token Response
[screenshots/task1/bug-02-api-cloudflare-530.png](#) — Bug 2 Production Backend API Cloudflare 530 Direct IP Access Failure
[screenshots/task1/bug-03-auth-header-mismatch.png](#) — Bug 3 Authorization Header Token vs Bearer 401 Rejection
[screenshots/task1/bug-04-empty-title-edit.png](#) — Bug 4 Blank Article Title Update & Corrupted Slug Generation
[screenshots/task1/bug-05-tag-overflow-sanitization.png](#) — Bug 5 Tag Pill Container Horizontal Overflow in Feed UI
[screenshots/task1/bug-06-expired-session-unhandled.png](#) — Bug 6 Stale JWT Expiration Infinite Loading Canvas State